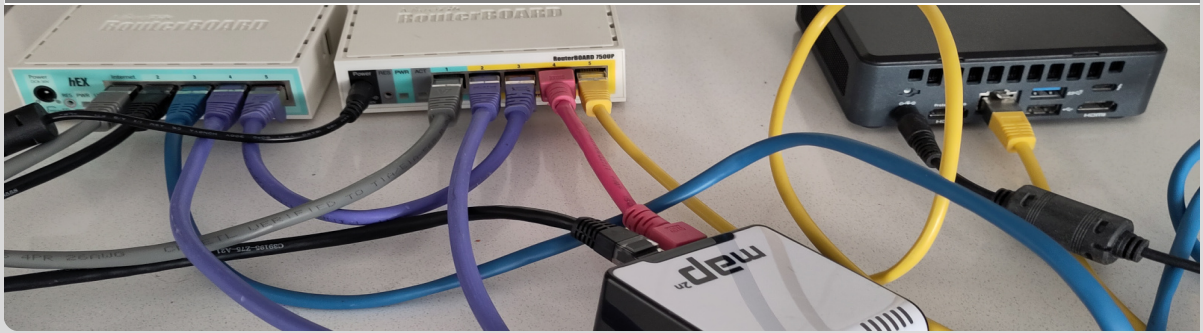
# Machine Learning for Cybersecurity (5883V/Ü)

3. More ML Basics

Hans P. Reiser | Winter term 2026/27

University of Passau



# Overview

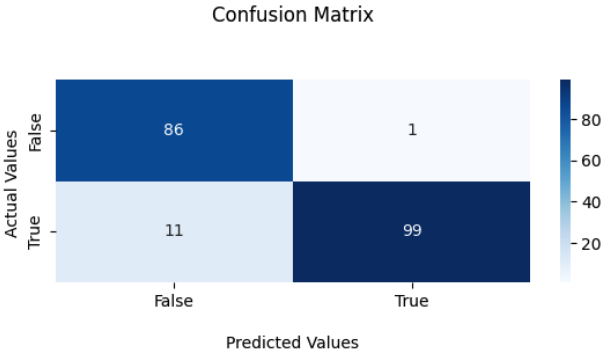
1. Metrics and Receiver Operating Curves
2. More ML basics
   - K Nearest Neighbors
   - Linear Regression and Logistic Regression
3. ROC revisited (with KNN)
4. Upcoming student presentations

# Evaluating the quality of a model

Let's take a closer look...

# Confusion Matrix: The “full” picture

We have seen this before (lecture part 2):



# Accuracy, Precision and Recall

**Accuracy: Correct prediction / Total predictions**

*ACC = (TP + TN) / ( TP + TN + FP + FN)*

# Accuracy, Precision and Recall

**Accuracy: Correct prediction / Total predictions**

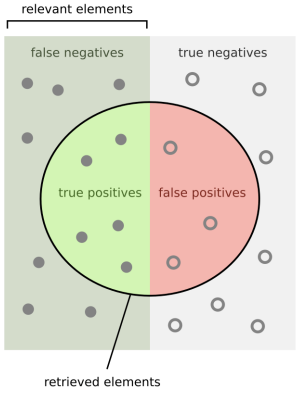
*ACC = (TP + TN) / ( TP + TN + FP + FN)*

**Precision (aka positive predictive value):**

*Precision = TP / (TP + FP)*

**Recall (aka sensitivity or TPR):**

*Recall = TP / (TP + FN)*




(source: Wikipedia)

# Accuracy, Precision and Recall

**Accuracy: Correct prediction / Total predictions**

*ACC = (TP + TN) / ( TP + TN + FP + FN)*

**Precision (aka positive predictive value):**

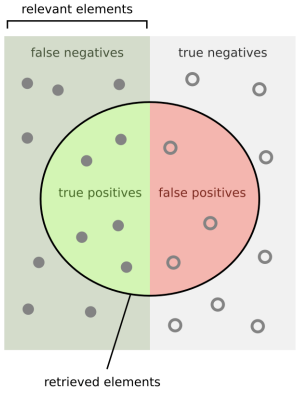
*Precision = TP / (TP + FP)*

**Recall (aka sensitivity or TPR):**

*Recall = TP / (TP + FN)*

**F1-Score: Harmonic mean of precision and recall:**

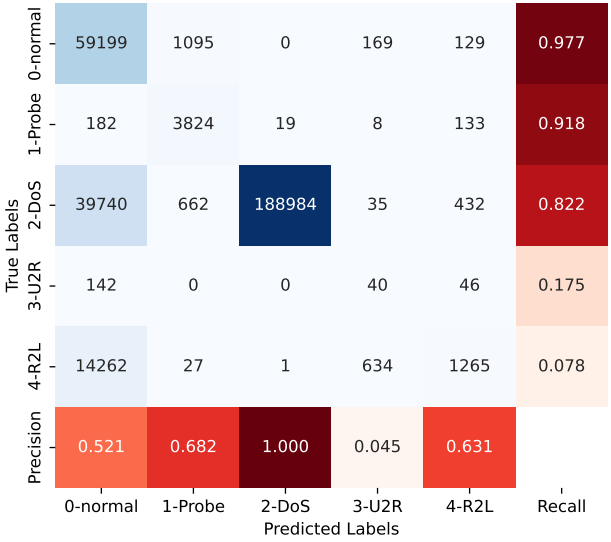
$$F1 = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}}$$




(source: Wikipedia)

# Confusion Matrix: The “full” picture

Can also be used with multi-class classification:



# Metrics from Credit Card example

```
Accuracy = 0.959464204206313
Precision = 0.03926482873851295
Recall = 0.912621359223301
F1 Score = 0.07529034841810173
              precision    recall  f1-score   support

           0       1.00      0.96      0.98     56859
           1       0.04      0.91      0.08       103

    accuracy                           0.96     56962
   macro avg       0.52      0.94      0.53     56962
weighted avg       1.00      0.96      0.98     56962
```

# Metrics from Credit Card example

Support: Number of samples for this class (in test set)

Macro average: Direct average of metrics of each class

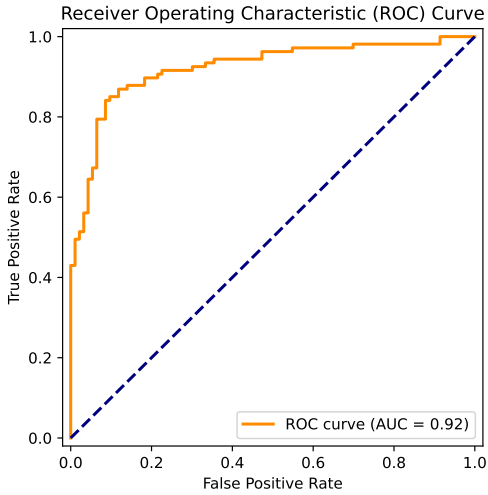
Weighted average: Metrics weighted by support

# Receiver Operating Characteristic (ROC) Curves

- Background:
  - “Signal Detection Theory” developed during World War II
  - Task: Radar operators had to classify observed radio signals into “enemy target” vs. “noise”
  - Measurement of the ability of *radar radio operators*
  - Later ($\approx$ 60ies) growing popularity in other fields, e.g. medicine
  - Today: Important measure for performance of a classification model

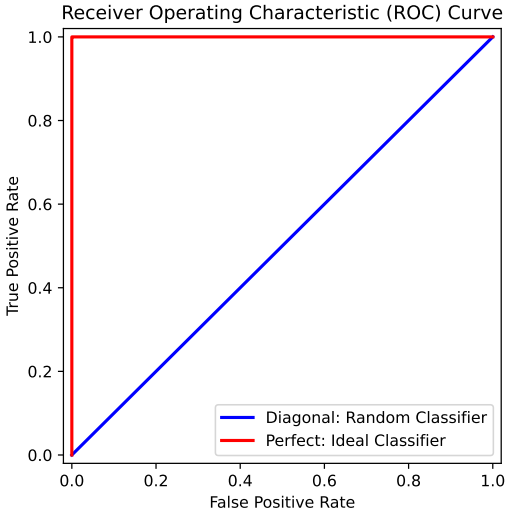
# Receiver Operating Characteristic (ROC) Curves

- Idea: Plot TPR vs. FPR
- FPR: False Positive Rate (FP/(FP+TN))
- TPR: True Positive Rate (TP/(TP+FN))
- . . . depending on some parameter (threshold)



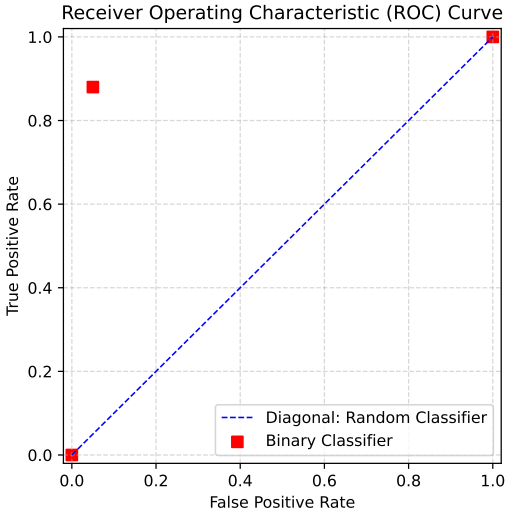
# Receiver Operating Characteristic (ROC) Curves

- The border cases: random and perfect
- Perfect: Actually just a single point with TPR=1, FPR=0
- Edge cases: Set parameter such that
  - All classified as 0: TPR=0, FPR=0
  - All classified as 1: TPR=1, FPR=1



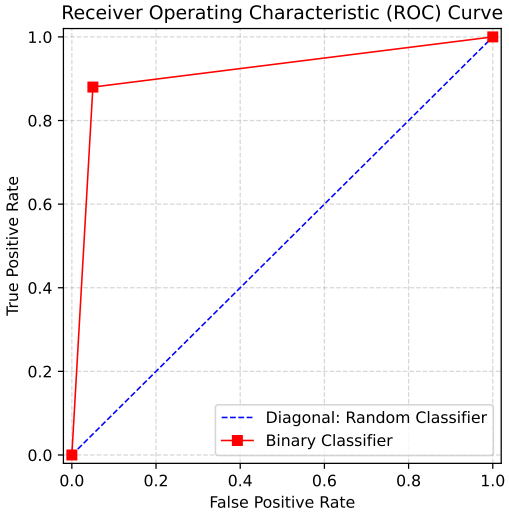
# Receiver Operating Characteristic (ROC) Curves

- Binary classifier on a ROC curve:
- For all samples, classifier produces 0 or 1 (Yes or No, Spam or Ham, etc.)
- How to parametrize with a “threshold”?
  - Either use classifier result
    $\Rightarrow$ a single data point on the diagram with FPR, TPR of the classifier
  - Or “ignore” the actual classifier and
    - Assume “0” for all: TPR=0, FPR=0
    - Assume “1” for all: TPR=1, FPR=1



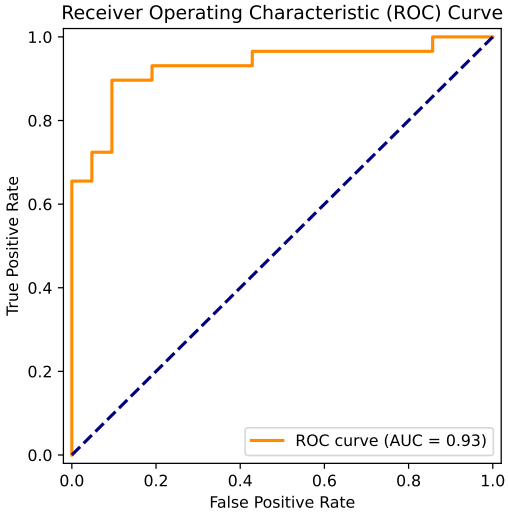
# Receiver Operating Characteristic (ROC) Curves

- Or, if you connect the dots with a line
- Of course, this is not very useful (no information gain compared to a simple confusion matrix or just the FPR and TPR values)
- What you usually want to plot: **probabilistic** classifier (each classification is a probability), you adjust a **threshold** (threshold: value above which a class probability is mapped to “1” (“true”))



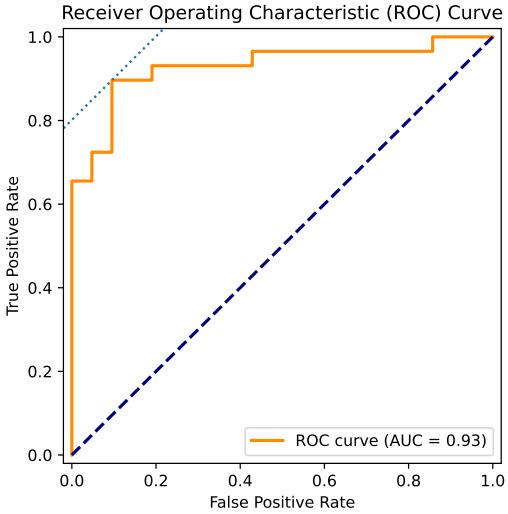
# Receiver Operating Characteristic (ROC) Curves

- Probabilistic classifier
  $\rightarrow$ you get a step function
- Why? Decreasing the threshold across some sample will give you an additional positive
  - An additional TP $\Rightarrow$ TPR increase
  - An additional FP $\Rightarrow$ FPR increase
  - (or both, if multiple samples with exactly the same probability score exist)



# Receiver Operating Characteristic (ROC) Curves

- Threshold optimization:
  - If FPR and TPR equally important:
  - i.e. maximize TPR + (1-FPR):
  - Optimum == point where line with slope +1 touches ROC curve
  - Different weights FPR vs. TPR: line with different slope
- Area under curve (AUC) as measure (larger=better, independent on slope)



# Overview


1. Metrics and Receiver Operating Curves

2. More ML basics
   - K Nearest Neighbors
   - Linear Regression and Logistic Regression

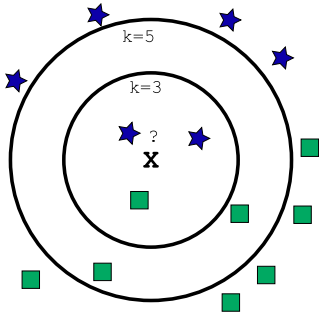
3. ROC revisited (with KNN)

4. Upcoming student presentations

# Classification with KNN (k nearest neighbors)

- *"Tell me about your friends (neighbors) and I will tell you who you are"*

- Simple but powerful data mining algorithm:
  - Classification of a new sample based on a similarity metric (*distance function*)
  - Object is classified by the most common class among its $K$ nearest neighbors (nearest = smallest distance)



# Classification with KNN (k nearest neighbors)

- Needed: Some “distance” metric (for multidimensional feature vectors)
  - $X = [x_1, x_2, \ldots, x_n]$, $Y = [y_1, y_2, \ldots, y_n]$
  - Euclidean: $D(X, Y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$
  - Manhattan: $D(X, Y) = \sum_{i=1}^{n} |x_i - y_i|$
  - Minkowski: $D(X, Y) = \sqrt[q]{\sum_{i=1}^{n} |x_i - y_i|^q}$ (generalization of above)

# Classification with KNN (k nearest neighbors)

How it works

- Training: actually, nothing to do, just collect training data

- Classification of new sample S:
  - Calculate distance to all elements of training data set
  - Find $K$ nearest elements
  - Assign the most frequent class in that set to S

- How to choose $K$?
  - Very small $K$: sensitive to noise (small random outliers) in training data
  - Very large $K$: might include majority of other class
  - Common rule of thumb: Choose $K$ as $\sqrt{N}$, with $N =$ number of samples in training set

# Classification with KNN (k nearest neighbors)

Extension: Feature weighting

- Idea: Scale each feature according to its importance (e.g., with Euclidean distance):

$$D(X, Y) = \sqrt{\sum_{i=1}^{n} w_i (x_i - y_i)^2}$$

- $w_i$: weight of feature $i$
- Choice of $w_i$ can incorporate knowledge about importance of features
- Choice of $w_i$ can be learned dynamically (e.g., using cross validation)

# Classification with KNN (k nearest neighbors)

Another important aspect: Feature normalization

- Distance dominated by features that are expressed by large numbers
- Important to normalize features

- Example from last week: scale to zero mean, unit variance

```
1 from sklearn.preprocessing import StandardScaler
2 s = StandardScaler()
3 x_train["Amount"] = s.fit_transform(x_train[["Amount"]])
4 x_test["Amount"] = s.transform(x_test[["Amount"]])
```

- Alternative: scale to the range $-1 \ldots 1$ (clip test values that fall outside)

```
1 from sklearn.preprocessing import MinMaxScaler
2 s = MinMaxScaler(feature_range=(-1, 1), clip=True)
3 x_train["Amount"] = s.fit_transform(x_train[["Amount"]])
4 x_test["Amount"] = s.transform(x_test[["Amount"]])
```

# Classification with KNN (k nearest neighbors)

Advantages of KNN

- Simple and intuitive
- Does not require any training phase
- Quite good classification (if number of samples is large enough)

Disadvantages of KNN

- Computational complexity of classification
- Sensitive to noisy data and outliers
- Choice of $K$ can significantly impact performance
- Distance becomes less meaningful for very high number of features (dimensions)
- Bias towards majority class in imbalanced data sets

# Overview


1. Metrics and Receiver Operating Curves
2. More ML basics
   - K Nearest Neighbors
   - Linear Regression and Logistic Regression
3. ROC revisited (with KNN)
4. Upcoming student presentations

# Linear Regression

Basic idea:

- You have a training set $\{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$
  ($\mathbf{x}_i$ is a d-dimensional feature vector)
- You want to have a function $y = f(\mathbf{x})$ that estimates (predicts) $y$ based on $\mathbf{x}$
- With $\mathbf{x} = [x_1, \ldots, x_d]$ and a **linear function** $f$: $f(\mathbf{x}) = b + w_1 x_1 + \ldots + w_d x_d$
- $w_i$: "weights", $b$: "bias" (model parameters)

- Simplified notation:

  Assume there is a constant $x_0 = 1$, and $w_0 = b$, i.e. $f(x_0, x_1, \ldots, x_d) = \sum_{i=0}^{d} w_i x_i$

- Or, in vector notation: $y = \mathbf{w}^T \mathbf{x}$

# Linear Regression

Goal: Find the **best** model!

- What is best? You need some evaluation criteria!
- Common choice: least squares, i.e. minimizing the sum of the squares of the residuals (residual: difference between observed value $y$ and predicted value $f(x)$)
- I.e., given the training set, minimize $E(\mathbf{w}) = \sum_{i=1}^{n} (y_i - \mathbf{w}^T \mathbf{x}_i)^2$
- On this basis, prediction is easy (just calculate $f(\mathbf{x})$), but...
- ... how do you find the best weights $\mathbf{w}$?

# Linear Regression

- Rewriting in matrix notation: assume that
  $X$ is the $n$ x $d$ matrix of training data
  $Y$ is the $n$ x 1 vector of output data
  $w$ is the $m$ x 1 vector of weights (as before)

- $f(X) = X\mathbf{w}$

$$\begin{bmatrix} x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} = \begin{bmatrix} x_{11}w_1 + x_{12}w_2 + \dots + x_{1n}w_n \\ x_{21}w_1 + x_{22}w_2 + \dots + x_{2n}w_n \\ \vdots \\ x_{m1}w_1 + x_{m2}w_2 + \dots + x_{mn}w_n \end{bmatrix}$$

- $E(\mathbf{w}) = (Y - X\mathbf{w})^T(Y - X\mathbf{w})$

- Minimum of $E$: derivative of $E$ is 0; $\partial E(\mathbf{W})/\partial \mathbf{w} = -2X^T(Y - X\mathbf{w})$

- $\Rightarrow X^T(Y - X\mathbf{w}) = 0$

# Linear Regression

- Solving $X^T(Y - X\mathbf{w}) = 0$
- $X^T Y = X^T X\mathbf{w}$
- $\mathbf{w} = (X^T X)^{-1} X^T Y$

- Looks very simple, but there is some computational cost:
  - $X^T X$: multiply $d$ x $n$ matrix with $n$ x $d$ matrix
  - $(X^T X)^{-1}$: inversion of a $d$ x $d$ matrix

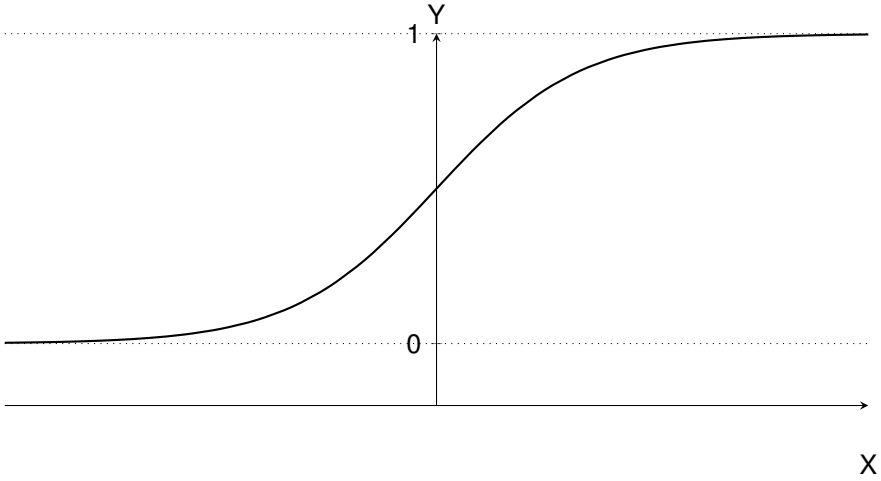
- Note: For very large datasets, iterative optimization algorithms like gradient descent can be more efficient
- E.g. Stochastic Gradient Descent in scikit-learn: **SGDRegressor**

# Linear Regression and Logistic Regression

- Linear Regression predicts a continuous value $y$ based on some input $x_1, x_2, \ldots, x_d$
- For classification, we want a categorical output (e.g., spam yes or no)
- Logistic Regression:
  - Passes the outcome of a linear function of features through a logistic function to calculate the probability of an occurrence
  - Maps the probability to binary outcomes
- Logistic function: Sigmoid

# Logistic Regression

- Sigmoid function: $f(x) = \frac{1}{1+e^{-x}}$



# Linear Regression and Logistic Regression

- The sigmoid function introduces non-linearity into the model
- As a result, we need a different cost function
  - Squared error function we used before is not suitable:
  - It will have multiple local minima, difficult to optimize
- Cost function: Log Loss
- Log Loss $= -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$,
  with $y_i$: true value (0 or 1), $\hat{y}_i$: predicted value
- No closed-form solution for the optimum, but several iterative optimization algorithms can be used (e.g., gradient descent)

  (see also `https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html`)

# Comparison

| Algorithm | Computational Effort (Training) | Computational Effort (Classification) | Dynamic Model Update |
|---|---|---|---|
| Naïve Bayes | Low | Low | Easy |
| KNN | Low | High | Easy |
| Logistic Regression | Moderate | Low | Complex (Online Learning) |

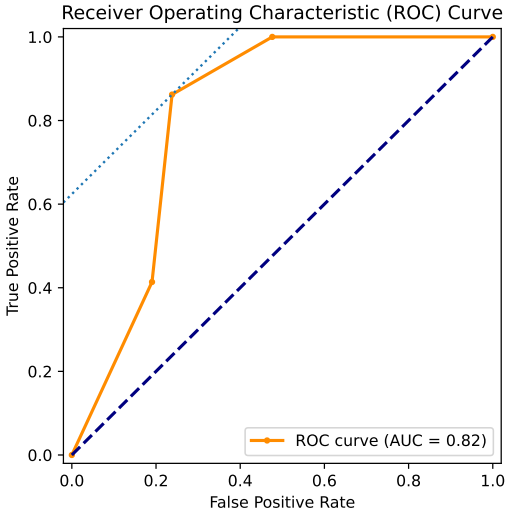
Note: While Naïve Bayes looks like the “best” approach in this table, it has also disadvantages. A core limitation is the assumption of independence of all features (which might not hold in real-world scenarios), and as a result it lacks the capability of capturing complex relationships between features.

# Overview


1. Metrics and Receiver Operating Curves
2. More ML basics
   - K Nearest Neighbors
   - Linear Regression and Logistic Regression
3. ROC revisited (with KNN)
4. Upcoming student presentations

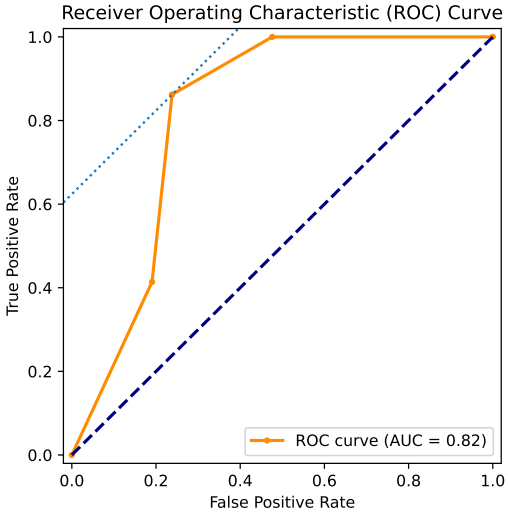
# Receiver Operating Curves

- ROC with 3-KNN classifier
  (will look like that, independent of number of samples in test set)
- Why no steps??



# Receiver Operating Curves

- ROC with 3-KNN classifier
  (will look like that, independent of number of samples in test set)
- Why no steps??
- No continuous probabilities:
  Can have 0, 1, 2 or 3 neighbors of a class
- Only 3 discrete TPR/FPR pairs:
  threshold between 0–1, 1–2, or 2–3



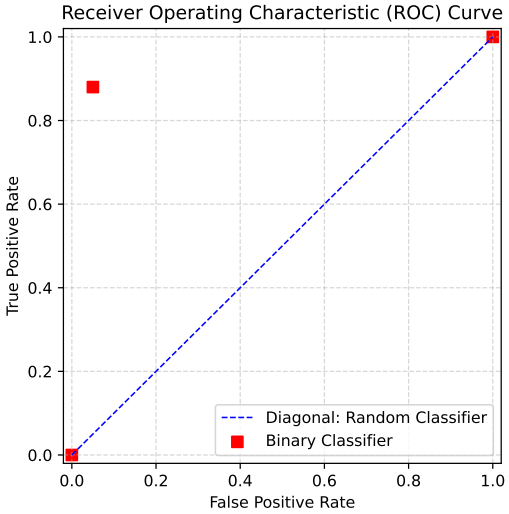
# Receiver Operating Curves

## Quiz question

How does the ROC curve of a 1-KNN classifier look like?

# Receiver Operating Curves

- 1-KNN: only 0 or 1 neighbor of a class
- Score is always 0 or 1
  (no intermediate probability)
- Only **one** operating point:
  the hard classifier itself
- Plus the two trivial corners
  (all 0 / all 1)
- Same as any non-probabilistic
  binary classifier



# Student presentations: Network intrusion detection

Select one of the following papers and present the key insights (20 minutes) on Nov 3 (or Oct 29).

- Apruzzese, Laskov, Schneider: “SoK: Pragmatic Assessment of Machine Learning for Network Intrusion Detection”, IEEE EuroS&P 2023.
  `https://arxiv.org/pdf/2305.00550`
- Fu, Li, Xu: “Detecting Unknown Encrypted Malicious Traffic in Real Time via Flow Interaction Graph Analysis” (HyperVision), NDSS 2023.
  `https://www.ndss-symposium.org/wp-content/uploads/2023/02/ndss2023_s80_paper.pdf`
- Jacobs, Beltiukov, Willinger, et al.: “AI/ML for Network Security: The Emperor has no Clothes”, ACM CCS 2022.
  `https://sites.cs.ucsb.edu/~arpitgupta/pdfs/trustee.pdf`

# References I


Giovanni Apruzzese, Pavel Laskov, and Johannes Schneider. SoK: Pragmatic assessment of machine learning for network intrusion detection. In *8th IEEE European Symposium on Security and Privacy (EuroS&P)*, 2023. 10.1109/EuroSP57164.2023.00042. URL https://arxiv.org/pdf/2305.00550.

Chuanpu Fu, Qi Li, and Ke Xu. Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis. In *Proc. Network and Distributed System Security Symposium (NDSS)*, 2023. 10.14722/ndss.2023.23080. URL https://www.ndss-symposium.org/wp-content/uploads/2023/02/ndss2023_s80_paper.pdf.

Arthur S. Jacobs, Roman Beltiukov, Walter Willinger, Ronaldo A. Ferreira, Arpit Gupta, and Lisandro Z. Granville. AI/ML for network security: The emperor has no clothes. In *Proc. ACM CCS*, 2022. 10.1145/3548606.3560609. URL https://sites.cs.ucsb.edu/~arpitgupta/pdfs/trustee.pdf.